2026-2 Midterm

Object-Oriented Programming (C++) · Midterm Study Notes

5 units · 23 lessons · same content as the trainer's Learn tab (English)

U1 Intro to SW Engineering and OOP

L1-1 · Software Crisis · Life Cycle · Software Engineering
L1-2 · Factors of Good Software · Software Properties
L1-3 · Programming Paradigms
L1-4 · OOP Concepts: ADT · Object · Encapsulation · Inheritance · Polymorphism

U2 C++ in Summary (Syntax · Pointers · File I/O)

L2-1 · Compiling · I/O · string
L2-2 · Pointers · Dynamic Allocation · Static vs Dynamic
L2-3 · C++ Class Basics · Pointers to Objects
L2-4 · File I/O

U3 Object-Oriented Design (IIKH)

L3-1 · Responsibility-Driven Design · Development Process · Scenarios
L3-2 · Identifying Components · CRC Cards · The Six IIKH Components
L3-3 · Characteristics of Components: Behavior/State, Coupling/Cohesion, Interface/Implementation
L3-4 · Formalizing the Interface · Names · Implementation · Integration (Stubs) · Maintenance

U4 C++ Data Abstraction: class

L4-1 · Data Abstraction · Encapsulation · Information Hiding
L4-2 · Behavior and State of a Class · Methods · Message Passing
L4-3 · Avoiding Public Data · Getters/Setters · Invariants and assert
L4-4 · PlayingCard Example · Visibility · Constructors · Default Constructor
L4-5 · Copy Constructor · Shallow/Deep Copy
L4-6 · Object Creation and Memory Problems
L4-7 · static · const Member Functions · inline · Separating Definition and Implementation
L4-8 · friend · References
L4-9 · Argument Passing Modes · Memory Safety · Returning Local References
L4-10 · Copy-On-Write · TIntStack

U5 Operator Overloading (Project 2 reference)

L5-1 · Operator Functions: Member Method vs Non-Member (friend) Method

U1 · L02 Intro to OOP.pdf + Handout 1

Intro to SW Engineering and OOP

Software crisis → software engineering → factors of good software → programming paradigms → OOP concepts. Question 1's blanks (software crisis, complexity, maintenance, global/dependent) come from here almost every year. Memorizing the exact English words is what earns the points.

Software Crisis · Life Cycle · Software Engineering

Slides: L02 p.3–6

§ Software Crisis

- Rapid increases in computer power and in the **complexity** of the problems (SW); software technology cannot follow the complexity
- The difficulty of writing correct, understandable, and verifiable computer programs
- Symptoms: unmanageable, difficult to maintain, **over-budget**, **over-time (late)**, poor quality, software often did not meet requirements
- **Cause: complexity**

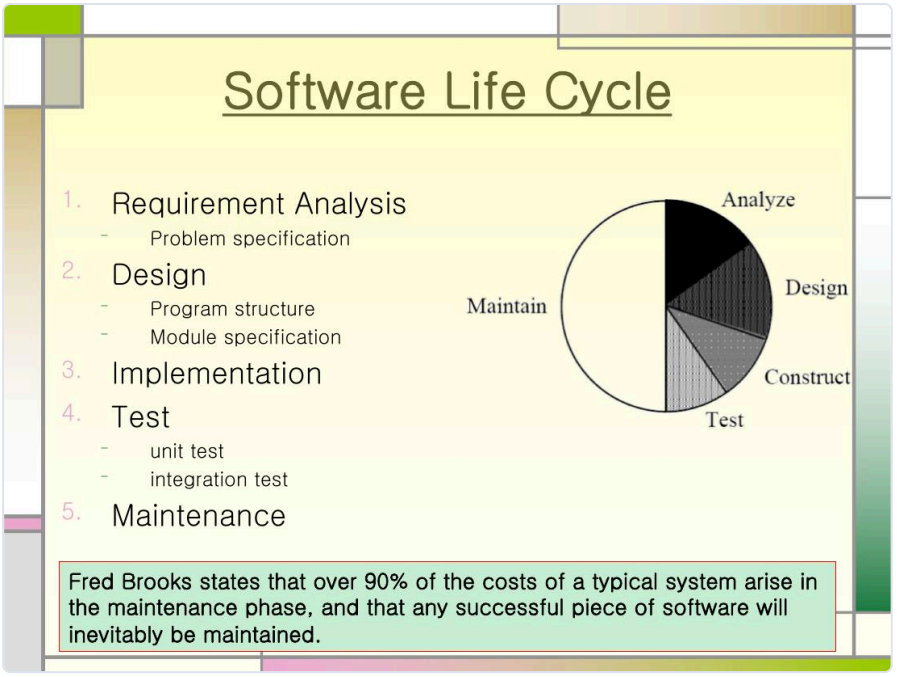
> Exam point: blanks in 2025·2023·2022. "(software crisis) arose because as computer hardware power grew rapidly, (complexity) of the software systems increased even faster".

§ Software Life Cycle

1. **Requirement Analysis**: problem specification
2. **Design**: program structure, module specification
3. **Implementation**
4. **Test**: unit test, integration test
5. **Maintenance**

Fred Brooks: over 90% of the **costs** of a typical system arise in the **maintenance phase**, and any successful piece of software will inevitably be maintained.

> Exam point: blanks in 2024·2017 "over 90% of (the costs) of a typical system arise in (the maintenance) phase, and any successful piece of software will inevitably be (maintained)".



L02 p.4 · Software Life Cycle

§ Definition and Goals of Software Engineering

Definition

- [IEEE] The application of a **systematic, disciplined, quantifiable** approach to the development, operation, and maintenance of software, that is, the application of engineering to software
- [Fritz Bauer] The establishment and use of sound engineering principles in order to **economically** obtain software that is reliable and works efficiently

Goals

- Produce **correct, efficient, and reliable** software
- **Minimize the cost** of software development and maintenance

Four goals of SE ("Goals of SE" slide)

Goal	Meaning
Maintainability	the ability to easily make changes, enhancements, or improvements
Dependability	the ability to **rely on** the software to function properly when needed
Efficiency	the ability for software to use computing resources effectively (mainly space and time)
Usability	the ability for the end user to easily and effectively put the software to proper use

Exam point: blanks in 2022·2019 "(software engineering) … to (economically) obtain software that is reliable and works efficiently", 2025 "minimize (the cost)", 2017 "(Dependability) means the ability to rely on the software to function properly when needed".

Factors of Good Software · Software Properties

Slides: L02 p.2, 7 + Handout 1

§ Factors of Good Software (Handout 1)

Factor	Meaning
correct (correctness)	satisfies the user requirements/specification without error
efficient (efficiency)	optimizes resource usage (space/time, HW utilization) to deliver fast response times
easy to maintain (maintainability)	easy to read (readability), easy to modify (modifiability), easy to extend (extensibility, scalability), easy to reuse (reusability), easy to debug/test (testability), modular (modularity), easy to port (portability)
stable, reliable, robust	stability, reliability, robustness
easy to use (usability)	easy to use, easy to run
secure (security), integrity	protects data against unauthorized access, corruption, or malicious exploits
interoperability	easily exchanges data with other systems via standardized protocols and interfaces
concise, complete, consistent, well-documented, well structured	conciseness, implements all required functionality, consistency (e.g. naming conventions), documentation, good structure

Exam point: you must be able to write the parts of maintainability in English: "easy to read, modify, extend, reuse, debug/test, port" (2019·2016 "A program with high maintainability is easy to (read), easy to (modify), easy to (extend)...").

§ Stability · Reliability · Robustness

- **stability**: keeps functioning correctly as conditions change and over time (if it works sometimes and fails other times, it is unstable)
- **reliability**: the ability to **consistently perform its required functions without failure** under specified conditions for a specified period
- **robustness**: keeps functioning under exceptional situations (invalid input, disasters)
- The three words mean almost the same thing; only the emphasis differs slightly

Trap: 2025 blank "(reliability) means the ability of the software to consistently perform its required functions without failure under specified conditions for a specified period" vs 2017 "(Dependability) means the ability to rely on the software to function properly when needed". Tell them apart by the definition sentence.

§ Software Properties

- **Modularity** – divide and conquer
- **Encapsulation** – hide the implementation
- **Localization** – collect similar things together
- **Abstraction** – simplifying complex reality

- **Uniformity** – make everything look similar
- **Completeness** – do everything required
- **Confirmability** – be able to prove that the software works properly

Programming Paradigms

Slides: L02 p.8–15

§ Unstructured Programming

- Consists of **only one main program**: a sequence of commands or statements
- **Only global data**
- Uses the **goto statement**
- Disadvantage: if the same statement sequence is repeated, it must be written out again every time
- **Spaghetti code**: code with a complex and tangled structure, with goto's jumping all over the place → almost impossible to understand, hard to follow the flow, difficult to modify

> Exam point: 2024 blank "(Unstructured programming) makes extensive use of GOTO statements... a program contains only a single code block".

§ Procedural and Modular Programming

Procedural programming — the abuses of spaghetti code led to structured programming languages

- Program = **a sequence of procedure calls**; procedure = a grouped sequence of statements
- More structured (reduces repetition) and more error-free
- **Structured Programming ≒ top-down approach ≒ divide and conquer ≒ functional decomposition**

Modular programming

- SW is composed of separate parts (**modules**); procedures of a common functionality are grouped together into modules
- The program uses modules through an **interface**: **interface is visible, implementation is hidden**
- Improves maintainability

> Exam point: 2022 blank "structured programming ≒ (top-down approach) ≒ (divide and conquer) ≒ functional decomposition".

§ Don't Use Global Variables

- **Using global variables is unsafe**
  - Sharing data (global variables) violates the principle of modular programming
  - Every module can access every global variable without restriction → no module can be developed or understood independently
  - It makes **all modules dependent on one another**
- Solution: **Modularized Data** — **localize** data inside the modules → modules become more independent of one another → safe

> Exam point: almost every year (2024·2022·2019·2017·2016)! "Using (global) variables is often unsafe because this makes all modules (dependent) on one another."

§ Problems of Procedural/Modular Programming and OOP

Problems of procedural/modular programming

- **Decoupled** (separation of) data and operations
- Doesn't work well in large systems: difficult to debug, extend, and maintain; code reuse limited
- Explicit creation and destruction

OOP

- Program = **a set of interacting objects**; each object keeps its own state
- **Combines data and operations into an object**
- Structured: "data + function = program"; OOP: "data + function = object, **object + object = program**"
- OOP does not view the problem domain as a flow of processing, but as **objects that communicate with each other through message passing**
- An object works as a **component** of software, and **the interface of an object should be well defined**

Why OO? Good for developing large software, maintainability, **reuse software, not rewrite** → increases productivity, quality, understandability, scalability and lifetime. SW **reliability also increases** because reusable modules are already well tested.

> Exam point: 2024 blank "(Object-oriented programming) is a programming paradigm based on the concept of (objects), which can contain data and code".

OOP Concepts: ADT · Object · Encapsulation · Inheritance · Polymorphism

Slides: L02 p.24–33

§ Abstract Data Type (ADT)

- **ADT**: a set of data variables and associated operations that are **precisely specified independent of any particular implementation**
- **ADT = data + operations (no implementation)**
- In OOP, we don't define data for defining procedures; we **define the procedures necessary for defining data**
- An ADT is a type that combines data and procedures. In C++, a **class** is used to define an ADT
- The user only needs to know **what** an ADT does, not **how** it does it (e.g. push/pop of a stack)

> Exam point: 2025 essay "ADT is called abstract because … (its data and operations are specified independently of any particular implementation)".

§ Class and Object

- **class**: defines the attributes (= **member variables**) and behaviors (= **member functions**) of an object
- **object**: **an instance of a class**. Consists of internal data, internal operations, and **interface operations**
  - Has identity, attributes, behavior (methods), and relationships with other objects
  - **Only class methods** can access the internal data of an object
  - **Message passing**: requesting some work from another object by calling a method
- **Class vs Object**
  - Each object generated by the same class is a **different object** (each gets its own memory space and address)
  - Objects created by the same class **share the class's methods**
  - An object can be dynamically constructed or destructed during run-time

> Exam point: 2016 blank "(An object) is an instance of a class".

§ Information Hiding · Encapsulation (Overview)

- **Encapsulation conceals the functional details** of a class from objects that send messages to it
- **Internal data of an object can be modified only through an interface**
- Object diagram: interface operations (specification) on the outside; data and internal operations (implementation) inside

> Exam point: 2025 blank "In the concept of information hiding, (internal data) of an object can be modified only through (an interface)". Details in U4.

§ Inheritance and Polymorphism

- **Inheritance**: a child class (sub class) inherits the attributes and behaviors of the parent class (super class) and can define its own. The child's properties should be mostly similar to the parent's. **Goal: object (class) reuse** → reduces the cost of large SW development
- **Polymorphism**: allows the same method (message) to have different meanings. The ability of objects of different types to respond to method calls of the same name, each according to an appropriate type-specific behavior
  - In C++: **virtual function, operator overloading, function overloading, template**
- Dynamic binding
- **Benefits of the object model**: builds a system that evolves over time (extendible, stable); thinking in objects and classes is easier for humans; separating the client and implementor prevents accidental damage through data encapsulation; reusability

Trap: inheritance and virtual functions come after the midterm (weeks 6–7 in the course schedule, not yet covered this year). Same as the "out of scope" mark on the dashboard.

U2 · C++ in Summary.pdf + static vs dynamic 토론 자료

C++ in Summary (Syntax · Pointers · File I/O)

A summary of C++ syntax. What shows up directly on the exam: the three meanings of pointer `*` and `&`, static vs dynamic allocation, fill-in-the-blank on the Rectangle pointer code, and file I/O (open modes, tellg/seekg, copying a binary file). The file-copy code had to be written in full in both 2024 and 2025.

Compiling · I/O · string

Slides: C++ in Summary p.2–14

§ Compiling and Standard I/O

- **Compiler**: converts high-level language code to machine language code. The C++ compiler generates an executable file (binary code) from C++ source code
  - `g++ -o TestProg TestProg.cpp` → `./TestProg`
- `#include <iostream>` (header file), `using namespace std;`. C code may be used in a C++ program
- **cout**: screen stream object (standard output). `cout << "Hello";` → prints the string Hello; `cout << Hello;` → prints the value of the variable Hello
- **cin**: keyboard stream object (standard input). `cin >> age;` → keyboard input goes into the variable (the arrow shows the direction of data flow)
- Formatting like printf: `std::format` (C++20), `std::print` (C++23)

```
double pi = 3.141592; int age = 21;
printf("pi=%7.2f , age=%d\n", pi, age);
cout << format("pi={:7.2f} , age={}\n", pi, age);   // pi=   3.14 , age=21
```

§ Data Types · string · Control Structures · Functions · Arrays

- Variable = a **named memory location** where a value is stored; must be declared before use
- char 1B, short 2B, int 4B ($-2^{31} \sim 2^{31}-1$), float 4B, double 8B. Use unsigned for non-negative values only
- **string**: a standard library class. Unlike C-style strings (char*), string **automatically manages memory** and provides many convenient functions (length, find, substr, replace, insert, erase, c_str, stoi, stod, to_string)
- `getline(cin, str)`: reads a line of text
- for-loop execution order: (1) initialization → (2) condition → body → (3) increment → (2) → body → ...
- break = exit the loop; continue = jump to the condition check of the next iteration
- Functions: local variables (including parameters) are created when the function is called and destroyed when it returns. Why use functions: remove repetition, reuse, top-down design, easier debugging, easier modification/extension, readability
- Array: a **fixed-size** collection of elements of the same type, stored contiguously in memory, fast random access `x[15]=3;`

Pointers · Dynamic Allocation · Static vs Dynamic

Slides: C++ in Summary p.23–24 + 토론 자료

§ Pointers

- Pointer = a variable that stores a memory **address**. Its size is always 8 bytes (64-bit) / 4 bytes (32-bit)

```
int *xPtr, x;   // xPtr is a pointer, x is an int
x = 5;
xPtr = &x;      // xPtr points to x (value of xPtr = address of x)
*xPtr = 10;     // set the value pointed to by xPtr to 10 → x becomes 10
cout << x;      // 10
```

Three meanings of `*`: (i) multiplication (binary) `z=x*y;` (ii) the value pointed to (unary, the-value-of / dereference) `*x=5;` (iii) pointer type declaration `int *x;`
Three meanings of `&`: (i) bitwise AND (binary) `z=x&y;` (ii) address operator (unary, address-of) `x=&y;` (iii) reference type declaration `int& y=x;`

> Exam point: question type like the 2022 final "Meaning of '*' in line 2? '&' in line 3?". Be able to answer in English which meaning each line uses (pointer type declaration / reference type declaration / address-of / dereference / multiplication / bitwise AND).

§ Dynamic Memory Allocation

- "dynamic" means "**... occurs at run-time (execution-time)**"
- `malloc` in C ⇔ `new` in C++, `free` in C ⇔ `delete` in C++

```
int *x = new int[10];   // C: x = (int*)malloc(10*sizeof(int));
...
delete[] x;             // C: free(x);
```

- A single object: `delete p;`, an array: `delete[] p;`
- **Unlike Java, C++ requires you to free memory yourself** (no garbage collection)

> Exam point: 2019 blank "Dynamic memory allocation means memory allocation occurs at (run-time)".

§ Static vs Dynamic Allocation (discussion handout)

`int x[10];` (static) vs `int* y = new int[n];` (dynamic)

	Advantages	Disadvantages
Static	simple and fast; memory is **deallocated automatically** (on the stack, when it goes out of scope)	size is fixed at **compile time** → cannot change while running (waste or shortage)
Dynamic	size can be decided at **run time** → uses exactly as much memory as needed	the programmer must **deallocate it manually (delete[])** → risk of **memory leak, dangling pointer**

Exam point: the 2023 essay came straight from this handout (advantages, disadvantages, reasons — 4 boxes). Key words = **compile time vs run time**, **automatic vs manual deallocation**.

C++ Class Basics · Pointers to Objects

Slides: C++ in Summary p.25–28

§ class · Constructor · Destructor (summary)

- **class**: a data structure type that can contain member data and member functions. Defines a new data type
- **Constructor/destructor**: member functions that are **automatically called when an object is created/deleted**
  - A local object is deleted when the function returns → its destructor is called then. The destructor's name is `~ClassName`
  - If you don't write a constructor, the compiler generates an empty default constructor. Same for the destructor
  - Uninitialized member variables hold **garbage values** → dangerous and unstable. Initialize them in the constructor

§ Pointers to Objects (Rectangle example)

```
class Rectangle {
  private:
    int width, height;
  public:
    Rectangle() { width=1; height=1; }
    Rectangle(int x, int y) : width(x), height(y) {}
    int area() { return width*height; }
};
int main() {
  Rectangle obj(3,4);
  Rectangle *foo, *bar, *baz;
  foo = &obj;                     // assign an address
  bar = new Rectangle(5,6);       // dynamic creation: new returns an address
  baz = new Rectangle[2];         // array: default constructor called twice → each 1×1
  cout << "obj's area: "    << obj.area()    << '\n';   // 12
  cout << "*foo's area: "   << foo→area()   << '\n';   // 12
  cout << "*bar's area: "   << bar→area()   << '\n';   // 30
  cout << "baz[0]'s area: " << baz[0].area() << '\n';   // 1
  cout << "baz[1]'s area: " << baz[1].area() << '\n';   // 1
  delete bar;
  delete[] baz;
  return 0;
}
```

- **Objects use `.`, pointers use `→`** (= `(*foo).area()`). The array element `baz[0]` is an object, so it uses `.`
- `foo = obj;` (wrong) → `foo = &obj;` (right). `bar.area()` (wrong) → `bar→area()` (right)

> Exam point: a variant of this code every year in 2025·2024·2023·2019·2017 (fill in the blanks, find 5 errors, write the output). Get `.` / `→` / `&` / `new` / `delete[]` exactly right line by line.

File I/O

Slides: C++ in Summary p.29–39

§ File Stream Classes and open

- **ofstream**: stream class to write on files / **ifstream**: stream class to read from files / **fstream**: read and write from/to files (`#include <fstream>`)
- File processing order: **open → read/write → close**
- Decide when opening: text vs binary; for writing, **append vs truncate (overwrite)**; file position (automatic vs user-controlled)
- `myfile.open(filename, mode);` or via the constructor `ofstream myfile("example.txt");`
- Check that it opened: `if (myfile.is_open()) { ... }`; close: `myfile.close();`
- Write text: `myfile << "This is a line.\n";`; read: `while (getline(myfile, line)) cout << line << '\n';`
- State checks: `bad()` read/write failed, `fail()` bad + format error, `eof()` end of file, `good()` everything OK, `clear()` reset the flags

> Exam point: 2019 blank "ofstream: write on files / (ifstream): read from files / (fstream): read and write".

§ File Open Modes

Flag	Meaning
`ios::in`	open for input (reading)
`ios::out`	open for output (writing)
`ios::binary`	open in binary mode
`ios::ate`	set the initial position at the **end of the file** (otherwise at the beginning)
`ios::app`	**all output operations are performed at the end of the file** → **appends** to the existing content
`ios::trunc`	if the file is opened for output and **already exists**, its previous content is **deleted** and replaced by the new content

- Combine several with **bitwise OR** `|`: `myfile.open("example.bin", ios::out | ios::app | ios::binary);`
- **Default modes**: ofstream = `ios::out`, ifstream = `ios::in`, fstream = `ios::in | ios::out`
- Opening for writing erases the existing file and writes from the beginning (the professor warned about this several times)

> Exam point: 2023 blanks "ios::app means all output operations are performed at the (end)", "(ios::trunc) means if the file is opened for output and already (existing), its previous content is deleted". app also in 2019.

§ File Position: tellg / seekg

- There is a separate **get position** (reading) and **put position** (writing)
- `tellg()` / `tellp()`: return the current get/put position

- `seekg(position)` / `seekp(position)`: move to an absolute position
- `seekg(offset, direction)`: `ios::beg` (from the beginning), `ios::cur` (from the current position), `ios::end` (from the end)

Getting the file size (a blank every year)

```
streampos begin, end;
ifstream myfile("example.bin", ios::binary);
begin = myfile.tellg();          // beginning position
myfile.seekg(0, ios::end);       // go to the end
end = myfile.tellg();            // end position
myfile.close();
cout << "size is: " << (end-begin) << " bytes.\n";
```

> Exam point: 2024·2023·2019 "Fill in the boxes to print the size". The three lines `begin = myfile.tellg();` → `myfile.seekg(0, ios::end);` → `end = myfile.tellg();`.

§ Binary Files and File Copy

- Text vs binary: writing 253 as text gives the characters '2','5','3' (3 bytes); as binary it is 11111101 (1 byte)
- Binary I/O uses `write(memory_block, size);` / `read(memory_block, size);` (memory_block is a `char*`)

Reading a whole file (from the slides)

```
ifstream file("example.bin", ios::in | ios::binary | ios::ate);
if (file.is_open()) {
    streampos size = file.tellg();     // because of ate, end position = file size
    char* memblock = new char[size];
    file.seekg(0, ios::beg);           // rewind to the beginning
    file.read(memblock, size);
    file.close();
    ...
    delete[] memblock;
}
```

File copy (code writing in 2024·2025): the code above + `ofstream out("des.bin", ios::out | ios::binary); out.write(memblock, size); out.close();`

> Caution: copying line by line with getline corrupts a binary file and loses points (per past-exam answer notes). Memorize the order `ios::binary`, `ios::ate` → `tellg` → `seekg(0, ios::beg)` → `read` → `write` → `delete[]`.

U3 · L03 Object-Oriented Design.pdf (Budd Ch.3)

Object-Oriented Design (IIKH)

Responsibility-driven design and the 9-step development process, the IIKH example, characteristics of components (coupling/cohesion, interface/implementation), integration and stubs, maintenance. Coupling/cohesion blanks appeared in 2024·2023·2022·2019·2016, and essays on stubs and "slowly integrated" are regulars too.

Responsibility-Driven Design · Development Process · Scenarios

Slides: L03 p.2–9

§ Responsibility Driven Design

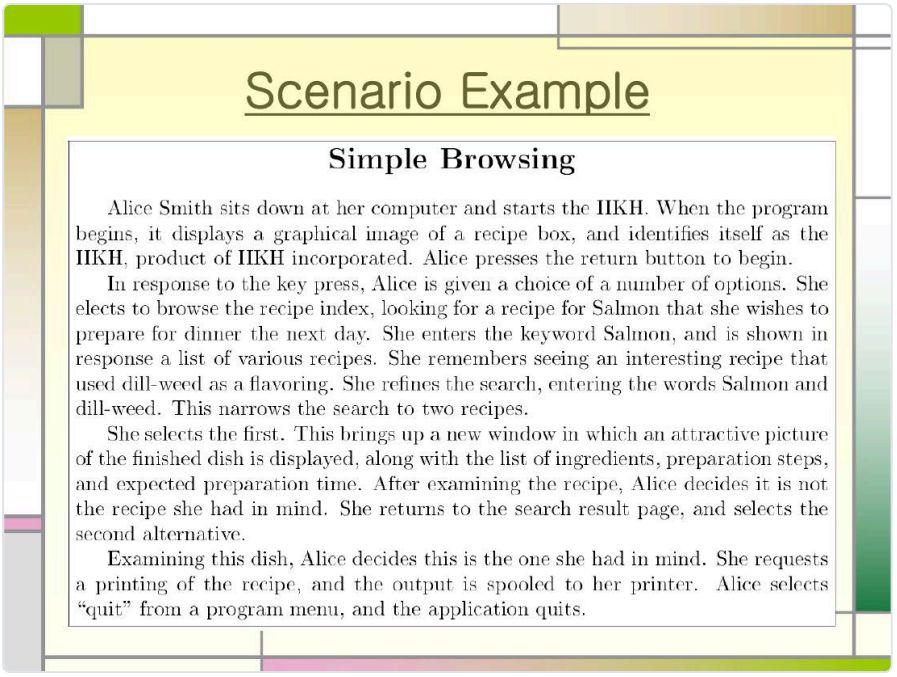
- OOP = a set of interacting objects
- Each object is a component that has **responsibility** in a community
- The system is divided into the various activities it has to perform; those activities (responsibilities) are then **distributed** among the objects in the system
- After completing a list of responsibilities, decide on a set of objects and determine which responsibilities should be assigned to which object
- **Responsibilities in an object should be closely related**
- Design process: an imprecise, ambiguous, unclear **specification** → a **software system design** with concrete logic

§ The 9-Step Development Process

1. **Initial Description** (Specification)
2. **Refine Specification** (scenarios)
3. **Identification of Components**
4. **Identification of Components Responsibilities** (CRC cards)
5. **Formalize the Interface**
6. **Designing the Subsystem**
7. **Implementing Components** (+ unit test)
8. **Integration of Components** (stubs)
9. **Maintenance & Evolution**

§ IIKH and Scenarios

- **IIKH (Intelligent Interactive Kitchen Helper)**: a PC-based application that replaces the box of recipe index cards in the kitchen. Maintains a recipe database + plans meals for an extended period (say a week), automatically scales recipes to any number of servings, prints menus for the week/day/meal, prints an integrated grocery list
- The initial description is **ambiguous and unclear** (the president drew it on the back of a dinner napkin) → the ambiguities must be clarified
- **Refining the specification = walking through scenarios**: **pretend** we already had a working application and walk through the various uses of the system
  - Establish the "look and feel", make sure we have uncovered all the intended uses, develop descriptive documentation, create the high-level software design
  - Other authors use the term **use-cases** (a set of scenarios of system usage)
- IIKH abilities: browse the recipe database, add a new recipe, edit or annotate a recipe, plan a meal of several courses, scale a recipe for some number of people, plan a longer period (a week), generate a grocery list



L03 p.8 · IIKH scenario example

Identifying Components · CRC Cards · The Six IIKH Components

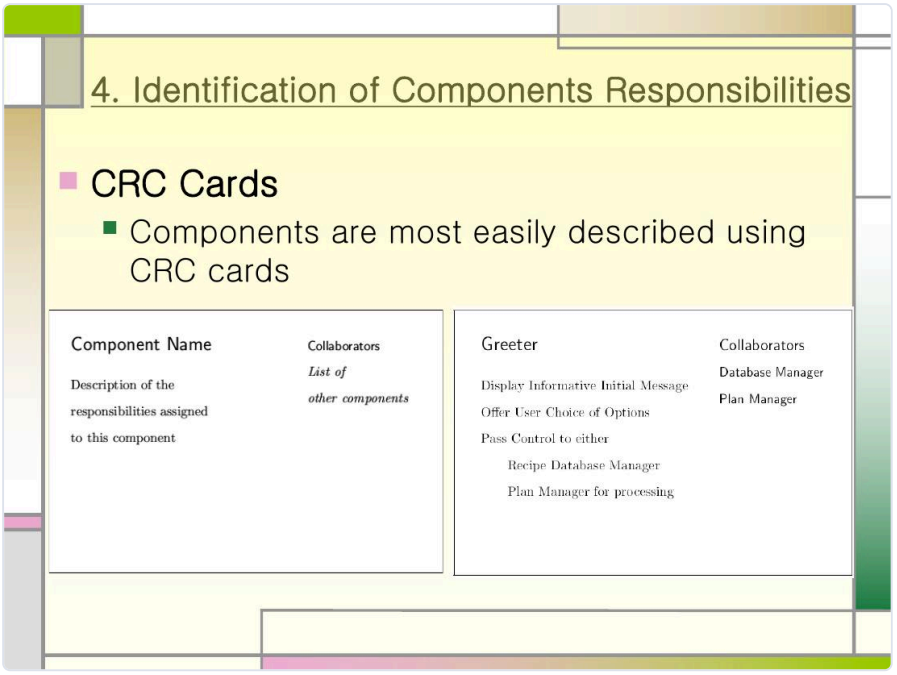
Slides: L03 p.10–18

§ Component

- **An abstract design entity** with which we can associate responsibilities for different tasks
- Must have **a small well-defined set of responsibilities**
- Should interact with other components **to the minimal extent possible**

§ CRC Cards

- Components are most easily described using **CRC cards**: **C**omponent (name), **R**esponsibility (description of its responsibilities), **C**ollaborators (list of other components it works with)
- e.g. **Greeter**: responsibilities = display an informative welcome message, offer the user choices, pass control to the Recipe Database Manager or the Plan Manager / collaborators = Database Manager, Plan Manager



L03 p.11 · CRC card format and the Greeter card

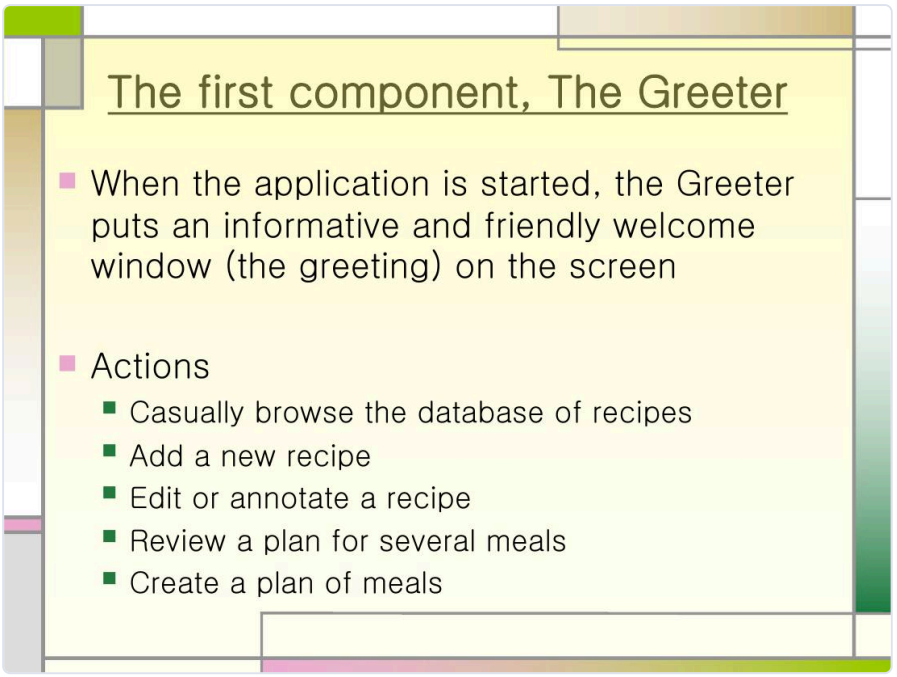
§ The Six IIKH Components

Component	Responsibilities
Greeter	welcome window at startup; lets the user browse the recipe database, add a new recipe, edit/annotate a recipe, review or create a meal plan
Recipe Database	maintains the recipe database, allows browsing, permits editing/annotating existing recipes, adding new recipes
Recipe	maintains the list of ingredients and the transformation algorithm, edits its own data, displays itself on the screen, prints itself

Component	Responsibilities
Planner (Plan Manager)	selects a sequence of dates for planning, edits an existing plan, associates with Date objects
Date	edits specific meals, annotates dates ("Bob's Birthday"), prints the grocery list for the entire set of meals
Meal	holds information about a single meal, interacts with the recipe database to select recipes, sets the number of people → recipes scaled automatically, grocery list for the meal

- Everything can be accomplished with only six components → at this point the components can be **assigned to different programmers** for development

> Exam point: 2025 essay "In project 1 (IIKH), main difference between Recipe class and RecipeDB class?" → Recipe maintains the data of **a single** recipe and edits/displays/prints itself; RecipeDB manages the **collection** of recipes (browse, search, add, edit).



L03 p.12 · The first component, the Greeter

Characteristics of Components: Behavior/State, Coupling/Cohesion, Interface/Implementation

Slides: L03 p.19–25

§ Behavior and State · Instances and Classes

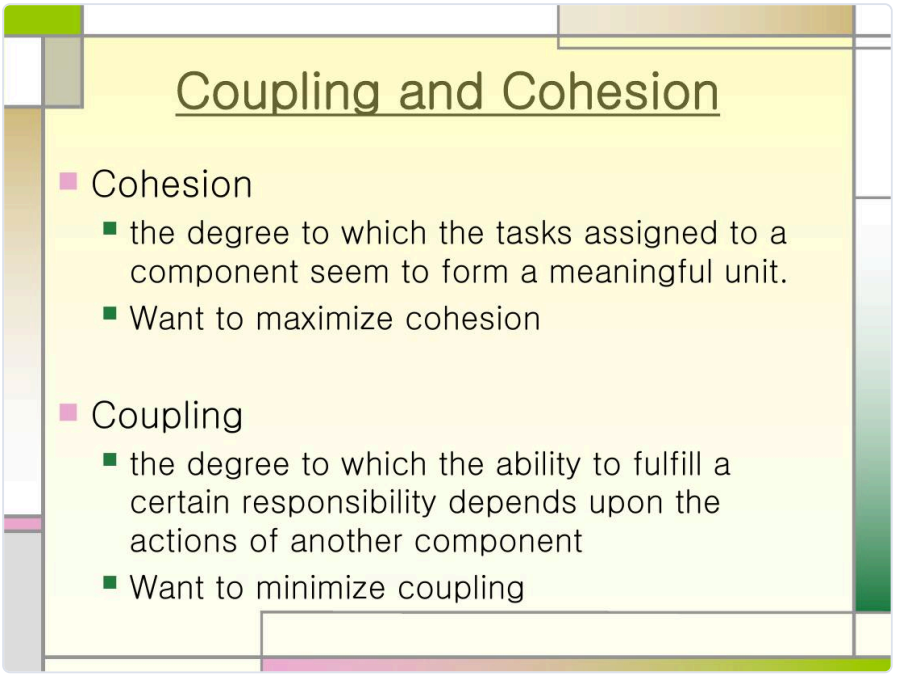
- **Behavior**: the set of actions a component can perform
- **State**: all the information (data values) held within a component. It is common for behavior to change state
- **Instances and Classes**: there are many instances of Recipe, but they all behave the same way → we say the behavior is **common to the class Recipe**

§ Coupling and Cohesion

	Definition	Goal
Cohesion	the degree to which the tasks assigned to a component seem to form **a meaningful unit** (how strongly its responsibilities are related)	**maximize**
Coupling	the degree to which the ability to fulfill a certain responsibility **depends upon the actions of another component** (interdependence between modules)	**minimize**

- Good design = **high cohesion, low (loose) coupling**
- With high coupling, a change in one module affects other modules → hard to maintain and reuse

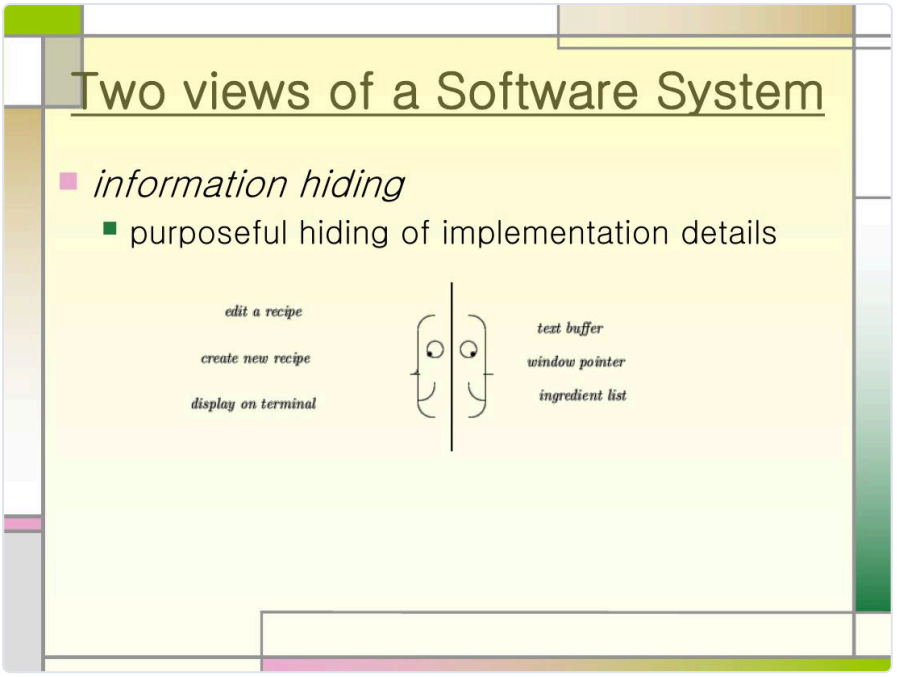
> Exam point: 2024 "(Coupling) and (Cohesion) are two key concepts used to measure design quality... good design = (high) cohesion, (low) coupling", 2023 "Coupling is the degree to which each module (relies/depends) on ... (Cohesion) ... (minimize) ... (maximize)", 2016 "(depends upon) ... desirable to (minimize)". Memorize the English definitions word for word.



L03 p.22 · Coupling and Cohesion

§ Interface and Implementation · Information Hiding

- We characterize software components by **what** they can do
- The user of a software component **need only know what it does, not how it does it**
- **Two views of a software system**: interface (edit a recipe, create new recipe, display on terminal) vs implementation (text buffer, window pointer, ingredient list)
- **information hiding**: the **purposeful hiding of implementation details**. A cover keeps you from seeing inside, which also gives **protection**



L03 p.24 · Two views of a Software System

§ Public View and Private View

- **Public view**: those features (data or behavior) that other components can see and use → the interface
- **Private view**: those features (data or behavior) that are used **only within the component** → the implementation
- Distinguishing public and private is a key feature of object-oriented programming (the professor stressed this)

Formalizing the Interface · Names · Implementation · Integration (Stubs) · Maintenance

Slides: L03 p.26–33

§ Formalizing the Interface and Choosing Names

5. Formalize the Interface: formalize the channels of communication between the components

- Identify the general structure of each component. Components with **only one behavior may be made into functions**; components with **many behaviors** are more easily implemented as **classes**
- Give a name to each responsibility → these eventually map onto procedure names. Identify function parameters, assign information to each component, replay the scenarios to ensure all data is available

Guidelines for selecting names

- **evocative** (in the context of the problem) — e.g. a name that reveals what is being counted rather than a meaningless name
- **short**
- **pronounceable** (read them out loud)
- **consistent** (within the project; e.g. if you decide variables start lowercase and functions start uppercase, keep it to the end)
- **avoid digits** within a name (count1, count2 tell you nothing about the difference)

> Exam point: 2016 essay "What does 'evocative' mean?" → The name should call to mind (suggest) the meaning/role of the thing in the problem domain.

§ Design · Implementation · Unit Testing

- **6. Design**: transform the description into a SW layout, design the data structures of each component, transform behavior descriptions into algorithms
- **7. Implementing Components**: implement the desired activities in each component. Components are divided among team members
  - **Unit testing**: subsystems are **validated individually**. Identify the necessary conditions for correct functioning and test with legal input values

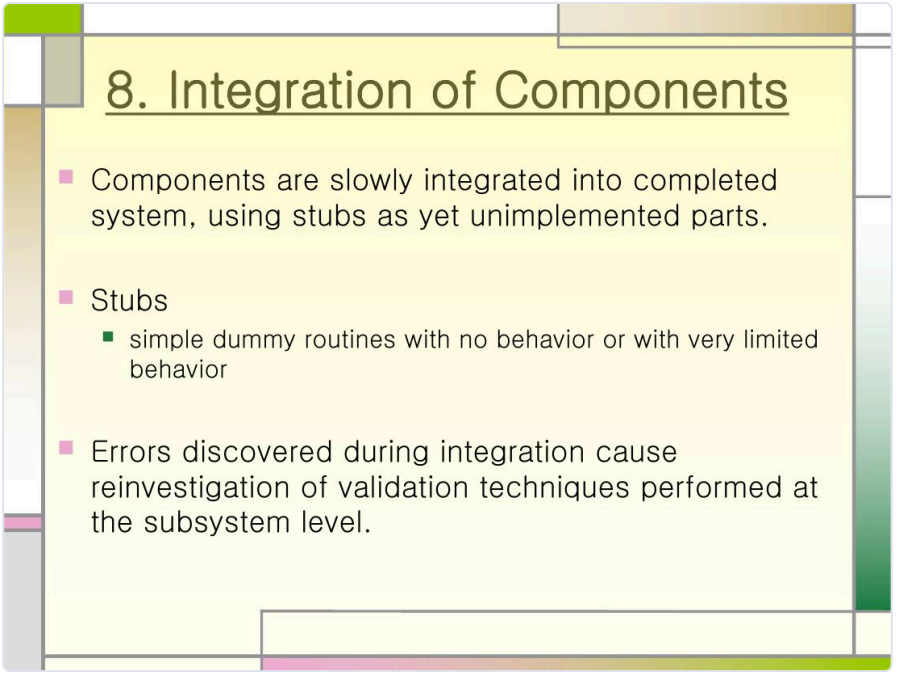
§ Integration and Stubs

8. Integration of Components: components are **slowly integrated** into the completed system, using **stubs** for as-yet unimplemented parts

- **stub**: a **simple dummy routine** with no behavior or with very limited behavior. A temporary stand-in for a component not yet developed
- **slowly integrated**: not integrated all at once, but **added one (or a few) at a time and tested** → if a problem appears, the part just added is the cause, so it is easy to find and fix. If everything is integrated at once, you cannot tell where the problem came from

- Errors discovered during integration cause reinvestigation of the validation techniques performed at the subsystem level

> Exam point: 2025·2016 essay "Meaning of 'slowly integrated'? Meaning of 'stubs'?", 2022 blank "(Stub) is a simple dummy routine with no behavior or with very limited behavior".



L03 p.30 · 8. Integration of Components (stub)

§ Maintenance and Evolution · Common Design Flaws

9. Maintenance and Evolution

- **Software maintenance**: activities **subsequent to the delivery** of the initial working version
- Software does not remain fixed after the first version: bugs are discovered and fixed, requirements change (government regulations, standardization), hardware changes, user expectations change (more features due to competing products), better documentation is required
- **Good design recognizes the inevitability of changes and plans an accommodation for them from the very beginning**
- **Prepare for Change**: predict the most likely sources of change and isolate their effect (interfaces, file formats, communication protocols), isolate interfaces to hardware likely to change, **reduce dependency of one component on another**, record the reasoning behind every major decision in the design documentation

Common Design Flaws

- **Direct modification**: directly modifying the data of another component → **a direct violation of encapsulation**
- **Too much responsibility**: difficult to understand and to use
- **No responsibility**: serves no purpose
- **Unused responsibility**: the result of designing components without thinking about how they will be used
- **Misleading names**: names should be short and unambiguously indicate the responsibilities

> Exam point: 2023 blank "(Software maintenance) is the process of changing, modifying, and updating software to keep up with customer needs. (It) is done after the product has launched".

U4 · L04 C++ Data Abstraction.pdf + Handout 2 + 보충 자료

C++ Data Abstraction: class

The professor called this unit "the most important part of this course". Data abstraction, encapsulation and information hiding, the three benefits of encapsulation, the three reasons to avoid public data, constructors, copy constructors, static, const member functions, inline, friend, references, and argument passing modes. The 10/7 class got as far as constructors, and there is no transcript of the 10/14 class. The material after that is also a past-exam regular, so it is covered here too.

Data Abstraction · Encapsulation · Information Hiding

Slides: L04 p.2–4 + Handout 2

§ The Three Terms

Term	Definition
Data Abstraction	creating classes with appropriate functionality, which hide the unnecessary details
Data Encapsulation	the implementation is **encapsulated** by the interface and **the only way to get to the implementation legally is through the interface**
Information Hiding	the implementation is **hidden** from the user

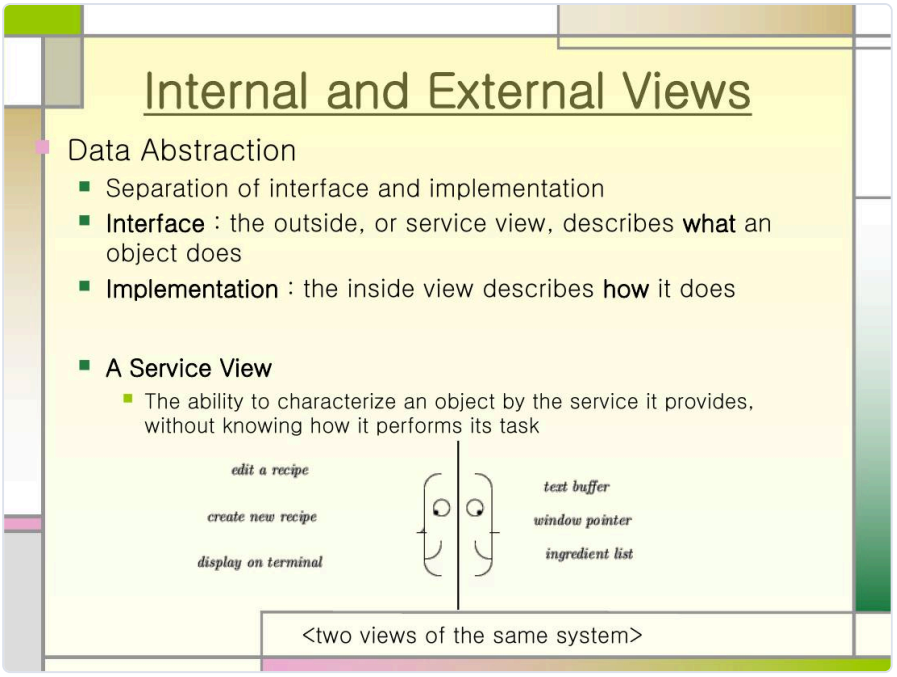
- The three terms are **almost the same concept** with slightly different emphasis (abstraction = show only the essentials, encapsulation = bundle data and functions and allow access only through the interface, hiding = hide it)

> Exam point: the professor said "this question also comes up in job interviews; this slide is the most important". Model answer = **"Separation of interface and implementation, such that the implementation is hidden from the user and the only way to access it is through the interface."**

§ Internal and External Views

- **Data abstraction = separation of interface and implementation**
- **Interface**: the outside, or **service view**; describes **what** an object does
- **Implementation**: the inside view; describes **how** it does it
- **Service view**: the ability to characterize an object by **the service it provides**, without knowing **how** it performs its task
- e.g. a projector: the buttons (screen up/down, on/off) = interface, the circuitry inside = implementation. The user only needs to know the buttons

> Exam point: 2019·2016 blanks "(Data abstraction) is a technique that relies on the separation of interface and implementation. Interface … describes (what) an object does. Implementation describes (how) it does".



L04 p.3 · Internal and External Views

§ Three Benefits of Data Encapsulation (Handout 2)

1. **Protection of implementation (safety)**: clients **cannot directly access** encapsulated data
2. **Readability (understandability)**: clients of the interface **do not need to know how** the functions that describe the interface use the encapsulated information
3. **Implementation independence**: **a change in implementation should not affect the interface seen by the clients** of a class. An encapsulated implementation gives the implementor **freedom to change it** without affecting the interface (how the clients use the interface is independent of the implementation)

Exam point: **the most frequently asked**. 2025·2022·2017 "List three main benefits ... explain each", 2023 blank "(implementation independence) means (a change) in implementation should not affect (the interface) seen by clients", 2019 "(1) Clients cannot (directly access encapsulated data) (2) Clients of interface does not need to (know how ...)". Memorize the three sentences in English as they are.

Behavior and State of a Class · Methods · Message Passing

Slides: L04 p.5–8

§ Behavior and State · The Three Parts of a Method

- **Class**: the fundamental unit of data abstraction in C++
- **Behavior (method, member function)**: the actions that an instance can perform in response to a request
- **State (variable)**: the data that an object must maintain in order to successfully complete its behavior. Stored in instance variables (data members, data fields)
- **The three parts of a method**
  - **Name**: will be matched to a message to determine when the method should be executed
  - **Signature**: **the combination of return type and argument types**. Methods with the same name can be distinguished by different signatures
  - **Body**: the code that will be executed when the method is invoked

§ Message Passing

Differences between a message and a function call

- A message is always given to **some object, called the receiver**
- The action performed in response is **determined by the receiver** → different receivers can do different actions in response to the same message

Three parts of message passing syntax: `aGame.displayCard(aCard, 42, 27);`

- **message receiver**: `aGame`
- **message selector**: `displayCard`
- **arguments** (optional): `aCard, 42, 27`

Avoiding Public Data · Getters/Setters · Invariants and assert

Slides: L04 p.9–12

§ Avoid Data Members in the Public Interface

Syntactically fine, but bad from an OOP point of view. Three reasons:

1. **Consistency**: a consistent policy of **always accessing member variables through the interface (functions)**
2. **Precise control over the accessibility of data members**: precisely control no access, **read-only, read-write, write-only**
3. **Functional abstraction**: you can later **replace the data member with a computation** (e.g. add logic such as counting the number of accesses)

```
class AccessLevels {
public:
  int  getReadOnly() const     { return readOnly; }
  void setReadWrite(int value) { readWrite = value; }
  int  getReadWrite() const    { return readWrite; }
  void setWriteOnly(int value) { writeOnly = value; }
private:
  int noAccess;   // no access to this int
  int readOnly;
  int readWrite;
  int writeOnly;
};
```

Exam point: 2024·2022 blanks "(i) (Consistency) (ii) precise (control) over the (accessibility) of data members (iii) functional (abstraction): you can later (replace) the data member with (a computation)". 2024·2022 essay "what if every member is private? → no interface, clients cannot use the object / what if every member is public? → no encapsulation, clients can directly access and corrupt data".

§ Accessor (getter) and Mutator (setter)

- **Accessor (getter)**: a method that **simply returns** an internal data value. Why use: makes the data field **read-only**, provides **better documentation** that the data field is accessible (visible from the interface alone), makes it easier to later change the access behavior (e.g. count # of accesses)
- **Mutator (setter)**: a method used to **change** a variable of an object. **Less common** than accessors (writing is risky, so keep it to a minimum where possible)

§ Class Invariants and assert

- **Class invariant**: a condition that **must always be true inside an object**. Add the condition so that every member function's code checks it
- The C/C++ **assert macro** `assert(e)` (`#include <assert.h>` or `<cassert>`)
  - **If e is true, just continue**
  - **If e is false, the program is stopped with an error message indicating the failure of the assertion**

- **Purpose: more reliable programs** → find and fix bugs early
- The professor: "put in as many asserts as possible"
- Trap: `assert(x = 3)` is an assignment, so it is always true (3 is non-zero). Comparison is `assert(x == 3)`

> Exam point: 2025 essay "(i) What does assert(e) do? (ii) main purpose? (iii)...", 2022 blank "(An assertion) contains a boolean expression that you believe will be true when it executes... writing (assertions) helps find (bugs)".

PlayingCard Example · Visibility · Constructors · Default Constructor

Slides: L04 p.13–16

§ The PlayingCard Class and Visibility

```
class PlayingCard {
public:
  enum Suits {Spade, Diamond, Club, Heart};
  enum Colors {Red, Black};
  PlayingCard ();
  PlayingCard (Suits is, int ir)                 // constructor
    { suitValue = is; rankValue = ir; faceUp = true; }
  ~PlayingCard () {}                             // destructor
  Suits suit () const { return suitValue; }      // getter
  int rank () const;                             // getter
  void setFaceUp (bool up) { faceUp = up; }      // setter
  Colors color ();
private:
  Suits suitValue;
  int rankValue;
  bool faceUp;
};
```

- color is not stored as data → it is **computed** from the suit (Red if Diamond or Heart)
- **Visibility modifier**: differentiates the interface and implementation views of a class
  - **public**: external (interface, service) view. Can be seen and manipulated by anybody (all clients)
  - **private**: internal (implementation) view. Accessible **only within the class** (only that class's member functions)
  - **protected**: discussed with inheritance (same as private when there is no inheritance)
- Design order: first decide the needed **interface (public)** → then put the functions and variables needed to implement it in **private**

§ Constructor

- A method that **initializes the data members** of a newly created object
- **Implicitly invoked when an object is created** (always)
- **No return type/value**
- **Has the same name as the class**
- There can be several (with different parameters)
- Initialization itself is not automatic → you must write initialization code in the constructor. Otherwise garbage values → dangerous

Member initializer

```
PlayingCard (Suits is, int ir) : suitValue(is), rankValue(ir) { faceUp = true; }
```

Exam point: 2019 blanks "(2) automatically called when (an object is created) (3) should always have the same (name) as the class (4) does not have (return) type", 2017 "automatically called when (an object) is (created)... the name is the same as (the class)... used to (initialize) data members".

§ Default Constructor and Destructor

- **If we do not declare any constructor in a class, the compiler generates a default constructor**
  - It looks like: `TIntStack::TIntStack() { /* Empty */ }` (no parameters, empty body)
  - **If you write even one constructor**, the default constructor is **not generated automatically**
- **Destructor**: called automatically when the object is deleted from memory. `~ClassName()`, only one. If you don't write it, the compiler generates an empty destructor

Exam point: 2025·2024 blank "if we do not declare any constructor in a class, (the compiler) generates (a default constructor)".

Copy Constructor · Shallow/Deep Copy

Slides: L04 p.17–18

§ When the Copy Constructor Is Called

Called **whenever a new object is created from an existing object of the same type**.

```
Person q("Mickey");   // q is built with a typical constructor
Person r(p);          // copy constructor is used to build r
Person p = q;         // initialization in a declaration → copy constructor
f(p);                 // initializing a value parameter → copy constructor
p = q;                // assignment operator! no constructor or copy constructor → p.operator=
(q)
```

- Declaration: `Person(const Person& p) { … }`
- Assignment operator: `Person& operator=(const Person& p) { … }`

> Exam point: 2017 "(a) Copy constructor is automatically called when (a new object is created from an existing object of the same type). (b) examples of the three cases" → `Person r(p);` / `Person p = q;` / `f(p);`. And know that `p = q;` is not one of them.

§ Shallow Copy and Deep Copy

- If you don't provide them, the compiler automatically generates a **default copy constructor, default assignment operator, and default destructor** → **shallow copy**: member values are copied as they are (**for a pointer, only the address is copied**)
- Don't write a copy constructor if shallow copies are OK
- If a pointer member points to dynamically allocated memory, a **deep copy** is needed: **allocate new memory with new and copy the contents**
- Problems with shallow copy: the two objects **share the same array** → changing one changes the other; when both are destroyed, **delete[] runs twice on the same memory** (double free, dangling pointer)

§ If You Need a Copy Constructor, You Also Need a Destructor and operator=

- **If you need a copy constructor, you also need a destructor and operator=** (Why? All three exist for the same reason — the object owns dynamically allocated resources: deep copy when copying, delete[] when destroying, free the old memory then deep copy when assigning)
- e.g. `inf_int` in Project 2: holds its digits through a dynamically allocated pointer → needs a copy constructor that does a deep copy. Without it, the shallow copy shares the array and delete[] runs twice

> Exam point: 2016 "Why do we generally need to write a copy constructor in class inf_int? What happens if we don't?", 2017 TIntStack "(4) What problems occur if the copy constructor is not provided?".

Object Creation and Memory Problems

Slides: L04 p.19

§ Object Creation and Deallocation

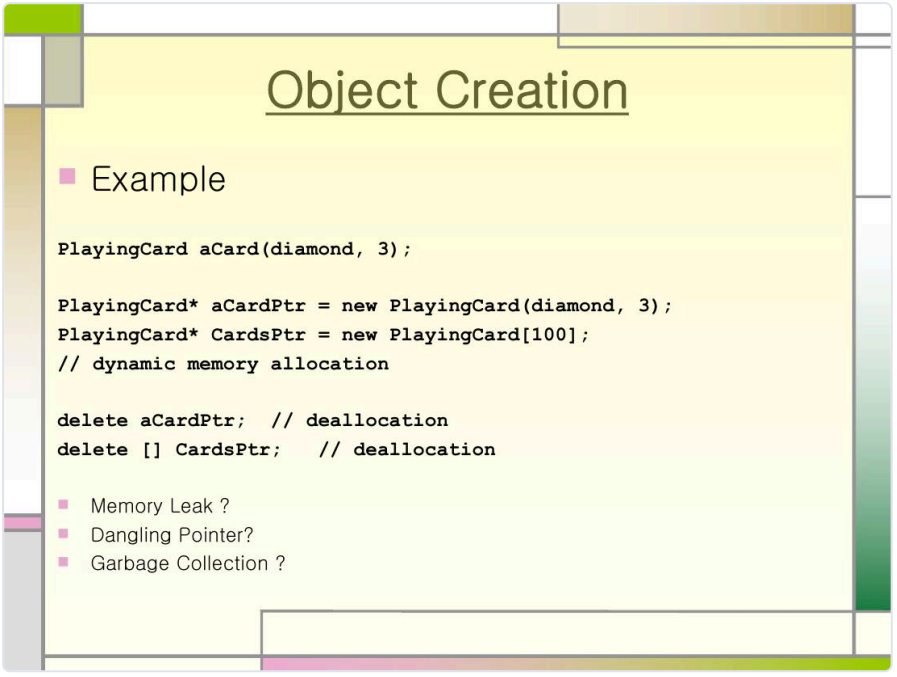
```
PlayingCard aCard(diamond, 3);                          // static (local object)
PlayingCard* aCardPtr = new PlayingCard(diamond, 3);    // dynamic
PlayingCard* CardsPtr = new PlayingCard[100];           // dynamic array (default constructor
100 times)
delete aCardPtr;      // deallocation
delete [] CardsPtr;   // array deallocation
```

- Local objects are destroyed **in reverse order of creation**; `delete[]` destroys array elements **from the last one backwards**

§ Memory Leak · Dangling Pointer · GC

- **Memory leak**: dynamically allocated memory that is **never deallocated** and can no longer be used (the pointer to it has been lost)
- **Dangling pointer**: **a pointer pointing to memory that has already been deallocated**. Accessing it is undefined behavior → crashes, corrupts memory used by other data
- **Garbage collection**: automatically reclaims unused memory (Java). C++ has none

> Exam point: 2025 essay "(i) What is a dangling pointer? (ii) Why dangerous?", 2024 blank "A pointer pointing to a memory location that has been deallocated is called a (dangling) pointer".



L04 p.19 · Object Creation (memory leak, dangling pointer)

static · const Member Functions · inline · Separating Definition and Implementation

Slides: L04 p.20–25 + 보충 자료

§ static Members

- **static**: all instances **share the same value**. **One per class**
- static members are used to represent **the properties of the class**
- **static member variable**: one per class, **accessible even if no object exists**
- **static member function**: can be called without an object as `ClassName::function()`; **cannot access non-static class data and functions**
- Defined and initialized outside the class: `int File::open_files = 0;`
- `static const int Spade = 1;` (const = it will not be reassigned)

```
class File {
  static int open_files;
public:
  File(const String& filename, const String& modes);
  File(const File&);
  ~File();
  static int existing() { return open_files; }   // number of existing File objects
};
int File::open_files = 0;
File::File(const String& filename, const String& modes) { ++open_files; }
File::~File() { --open_files; }
```

Exam point: 2025 blank "static members are used to represent the properties of (the class)... can be accessed even if there is no (created object)", 2024 "no matter how many (objects) are created, only one copy", 2022 essay on the File example "(3) What problem occurs without static?" → each object has its own open_files, each becomes 1, and the total number of File objects cannot be counted.

§ const Member Functions

- A member function that **does not modify any data members of the object on which it is called** (read-only function)
- This restriction is **enforced by the compiler**
- Benefit: assures its clients of its **benign nature** → safety, readability
- e.g. `int PlayingCard::rank () const { return rankValue; }`

Exam point: blank **four years in a row** (2025·2024·2023·2022) "(const) member function does not (modify) any data members of the object on which it is called. This restriction is enforced by (the compiler)".

§ inline Functions

(The slide only has the questions; this part was explained in class)

- **inline function**: a function whose **calls the compiler replaces with the function body (expands the code at the call site)**. A member function whose body is written inside the class definition is implicitly inline
- When it pays off: ① the body is **small** (getter/setter) ② it is **called very often** (inside a loop)
- Main benefit: **faster execution** — because the function call overhead (passing arguments, jumping, stack frame, returning) disappears
- Drawback: **larger code size (code bloat)**, especially when the function is large or called from many places

> Exam point: 2022 essay with 4 parts (definition, two situations where it helps, main benefit and why, drawback).

---

§ Order of Methods · Separating Definition and Implementation

Guidelines for listing methods: important topics first, constructors first, **public before private**, break long lists into groups, alphabetical order to make searching easier

Definition outside the class

```
PlayingCard::Colors PlayingCard::color ( ) {
  if ((suit == Diamond) || (suit == Heart)) return Red;
  return Black;
}
```

- Prefix with `ClassName::`. If the return type is a type declared inside the class, write it like `PlayingCard::Colors`
- Constructors and destructors have no return type: `TIntStack::TIntStack(...)`

friend · References

Slides: L04 p.26–31 + ref.cpp

§ friend

- **A friend function of a class has the right to access the non-public members** of the class
- To make all member functions of ClassTwo friends of ClassOne: put `friend class ClassTwo;` in the definition of ClassOne

```
class Count {
  friend void setX(Count&, int);   // friend declaration (may go in public or private)
public:
  Count() : x(0) { }
private:
  int x;
};
void setX(Count &c, int val) { c.x = val; }   // non-member, but may access private x
```

- **A member function** can access the private members of other objects of the same class too. Only **non-member functions** are blocked → the fix is one friend declaration line

> Exam point: 2023·2022 "code contains one compile error... (a) Which line? (b) Why wrong? (c) Insert one line to fix" → a non-member function accesses a private member → add `friend ReturnType functionName(args);` inside the class.

§ References

- **reference = alias for an object**
- Mainly used for **pass-by-reference**: avoids the run-time cost of passing large arguments by value
- Parameters, return values, and variables can all be defined as references

```
double square (double &x) { return x *= x; }
double foo = 10.0;
square(foo);   cout << foo;   // 100 (the original is changed)
```

- A function returning a reference → an lvalue: `v[3] = 100;` (= `v.operator[](3) = 100;`)

ref.cpp

```
int x=5;  int* y;  int& z=x;
y=&x;  int w=*y;  x=9;
z=*y*x;  x++;  z=x&*y;
cout << x << " " << *y << " " << z << " " << w << endl;   // 82 82 82 5
```

(z is an alias of x, y points to x → x=9 → z=9*9=81 → x++ = 82 → z = 82 & 82 = 82; w keeps the initial value 5)

§ Reference vs Pointer

- References are **implemented similarly to const pointers**

- Difference: **a reference must refer to an actual object**, but a pointer can refer to things that aren't objects (e.g. 0, NULL)
- In general, **references are safer, less ambiguous, and more restricted** than pointers

```
void printDouble(const double& rd) { cout << rd; }          // no need to test
void printDouble(const double *pd) { if (pd) cout << *pd; } // NULL check needed
```

Argument Passing Modes · Memory Safety · Returning Local References

Slides: L04 p.32–37

§ Argument Passing Modes (The Client's View)

```
1. void X::f(T arg)                // pass by value
2. void X::f(const T arg)          // pass by value
3. void X::f(T& arg)               // pass by reference
4. void X::f(const T& arg)         // pass by reference
5. void X::f(T* argp)              // pass by pointer
6. void X::f(const T* argp)        // pass by pointer
7. void X::f(T* const argp)        // pointer is constant
8. void X::f(const T* const argp)
```

- **Do not ever use pass by reference (without const qualifier) mode for read-only parameters**
- `T&` vs `T`: ① **avoids copying** (no copy constructor call → efficient for large objects) ② the function **can modify the original**
- `const T&` vs `T&`: avoids copying while guaranteeing **the original cannot be modified** (safe)

> Exam point: 2022 final essay "(1) Two purposes of '&' in (b) vs (a)? (2) Purpose of const in (c) vs (b)?".

§ Selecting the Correct Mode

- **Avoid passing large objects by value** — why? The whole object is **copied** into the parameter (copy constructor call, memory allocation and copying, destructor at the end), so the overhead is high. A reference passes **only the address**, so the overhead is very small
- For primitive types or small objects, pass **by value** (safer)
- Use **const** for arguments and functions wherever possible → a more robust program
- Don't pass pointers when you mean to pass objects `void f(T* object);` — a pointer can be NULL but there is no null reference; passing a reference does not transfer ownership of the object

> Exam point: 2017 essay "(a) Why does high overhead occur when passing large objects by value? (b) Why is the overhead very small with call-by-reference?".

§ Memory-Safe Classes · The Client's Responsibilities

Writing memory safe classes

- Avoid making copies of objects (expensive); avoid creating new objects and reuse existing ones
- Use const reference parameters and const member functions
- Use pointers instead of references as data members, delay storage allocation in the default constructor, use reference counting wherever possible
- Resist tricks in the initial stage: **safety first** (no memory leaks), optimization later

Client's responsibilities (summary): understand the purpose of the class, understand what the implementor expects (the contract), pay attention to member functions especially const ones, understand the arguments,

understand your responsibility for returned pointers/references, don't rely on undocumented details, prefer functions that take const pointers/references

§ Never Return a Reference to a Local Object

```
int  f () { int i = 2; return i; }    // OK (value is copied)
int* g () { int j = 2; return &j; }   // warning: address of local variable returned
int& h () { int k = 2; return k; }    // warning: reference to local variable returned
```

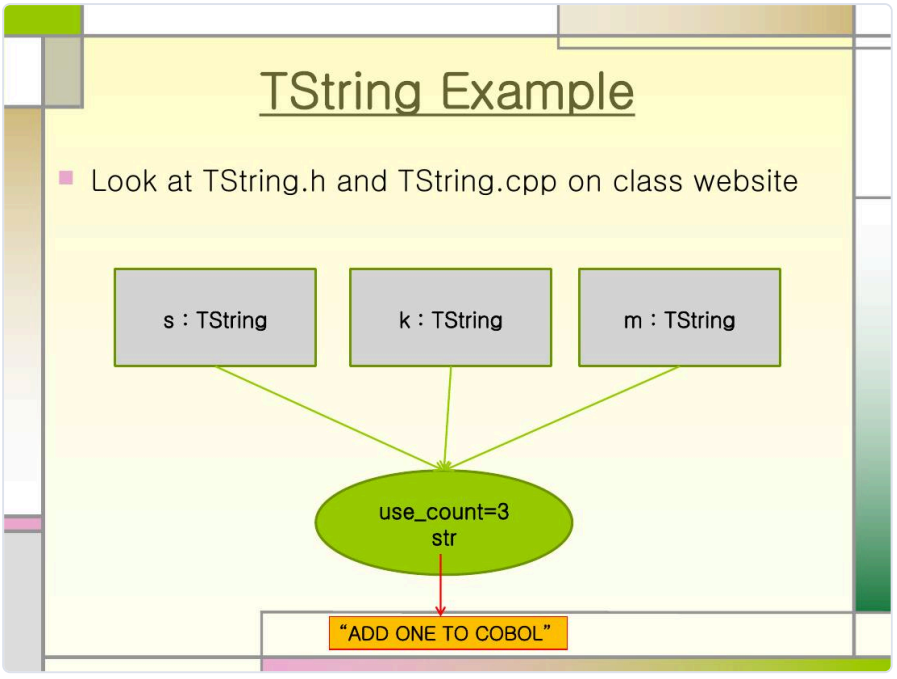
- When the function returns, its local variables are gone → that address/reference is **dangling** → undefined behavior

Copy-On-Write · TIntStack

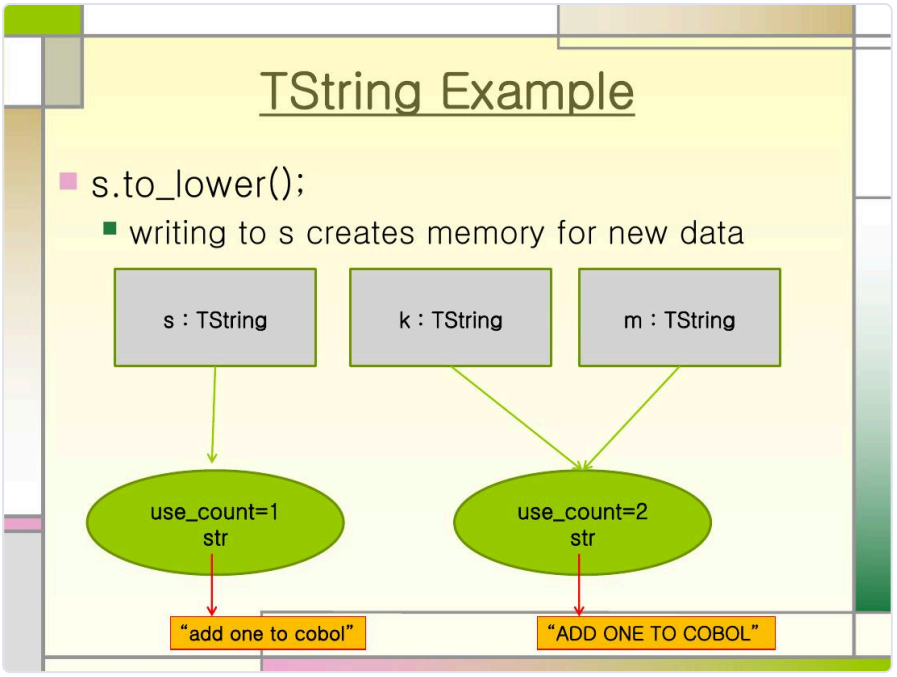
Slides: L04 p.38–40 + 기출

§ Copy-On-Write (TString)

- Object creation and destruction is expensive. If the copy constructor of a large object is called often, a lot of time goes into copying and deleting dynamic memory
- **Copy-on-write**: allocate new memory **only when writing**. Until then the original and the new object **share** the data
- Maintain a **reference count (use_count)**
- e.g. three TStrings s, k, m share one "ADD ONE TO COBOL" (use_count=3) → the moment `s.to_lower();` writes to s, only s gets new memory ("add one to cobol", use_count=1); k and m keep sharing (use_count=2)



L04 p.39 · TString: three objects share one data block (use_count=3)



L04 p.40 · After s.to_lower(): only s gets new memory

§ TIntStack (Deep Copy Implementation)

An integer stack built on a dynamic array. Past exams asked you to write the constructor, copy constructor, destructor and Push yourself (2022, 2017).

```
class TIntStack {
public:
  TIntStack(unsigned int stackSize = DEFAULT_SIZE);
  TIntStack(const TIntStack& that);
  ~TIntStack();
  bool Push(int thisValue);           // false if full
private:
  int* _sp;              // dynamically allocated in the constructor
  unsigned int _count;   // number of elements
  unsigned int _size;    // capacity
};
TIntStack::TIntStack(unsigned int stackSize) { _size = stackSize; _count = 0; _sp = new
int[_size]; }
TIntStack::TIntStack(const TIntStack& that) {          // deep copy
  _size = that._size; _count = that._count;
  _sp = new int[_size];
  for (unsigned int i = 0; i < _count; i++) _sp[i] = that._sp[i];
}
TIntStack::~TIntStack() { delete[] _sp; }
bool TIntStack::Push(int thisValue) {
  if (_count ≥ _size) return false;
  _sp[_count++] = thisValue;
  return true;
}
```

- Definitions outside the class: **no return type** (constructors/destructor), prefix with `TIntStack::`, access the other object with `.` as in `that._size`
- Writing only `_sp = that._sp;` is a shallow copy → points deducted

- Stack = **LIFO** (last in, first out): push 1, 3, 4 in that order and pop gives 4, 3, 1

U5 · Operator Overloading (Proj2 참고).pdf + Project2/src/complex.cpp

Operator Overloading (Project 2 reference)

Material for Project 2 (inf_int). The project deadline (11/1) is after the exam, so a coding question is unlikely, but the difference between the member-function method and the friend (non-member) method may come up as a concept question.

Operator Functions: Member Method vs Non-Member (friend) Method

Slides: Operator Overloading p.1–3

§ Operator Functions and the Two Methods

- **Operator function**: **redefines an operator** (`+ - * / [] << >>` ...) **as a function**
- **Member function method**
  - Uses `this` to implicitly get one argument → the **left operand** of a binary operator
  - The **leftmost object must be of the same class** as the operator
  - `c = a + b;` → `c = a.operator+(b);`
- **Non-member function method**
  - Needs **parameters for both operands**
  - The left object can be of a **different class** (e.g. `cout << obj`)
  - Must be a **friend** to access private/protected data
  - `c = a + b;` → `c = operator+(a, b);`

> Exam point: "In the member method, what provides the left operand? (this)", "What must a non-member operator function be to access private data? (a friend)".

§ Complex Example

Non-member (friend) method

```
class Complex {
private:
  double re, im;
public:
  Complex(double r, double i) : re(r), im(i) {}
  friend Complex operator+(Complex&, Complex&);
  friend Complex operator-(Complex&, Complex&);
  friend Complex operator*(Complex&, Complex&);
  void Display() { ... }
};
Complex operator+(Complex& i, Complex& other) {
  return Complex(i.re + other.re, i.im + other.im);
}
Complex operator*(Complex& i, Complex& other) {
  return Complex(i.re*other.re - i.im*other.im, i.re*other.im + i.im*other.re);
}
```

Member method

```
class Complex {
  ...
  Complex operator+(Complex& other);
};
Complex Complex::operator+(Complex& other) {
  return Complex(re + other.re, im + other.im);
}
```

- a = 1.2+3.4i, b = 5.6+7.8i → a+b = 6.8+11.2i, a−b = −4.4−4.4i, a×b = −19.8+28.4i (both methods give the same results)
- Project 2's `inf_int` uses the same approach to build an infinite-precision integer calculator supporting `+ - * ( )`. Because `inf_int` allocates its digits dynamically, it needs a **copy constructor, destructor and operator=** (L4-5)